CO3 AT3
Data Interpretation
PPO, TRPO & DDPG Performance Analysis

Name: P. Hithaishi
Reg no: 192425196
Course Code: MLA0305
Course name: Reinforcement Learning

---

1. PPO

PPO (Proximal Policy Optimization) is a policy optimization algorithm that improves the policy while preventing very large policy updates.

Main features:

- Stable training
- Relatively simple
- Good performance
- Uses a clipping mechanism
- Can work with discrete and continuous action spaces

In simple words:
PPO learns a better policy while making sure each update is not too large.

2. TRPO

TRPO (Trust Region Policy Optimization) is a policy optimization algorithm that restricts the change between the old and new policies.

Main features:

- Stable policy updates
- Uses a trust-region constraint
- Reduces the possibility of destructive policy updates
- More computationally expensive than PPO

In simple words:
TRPO makes sure that the new policy does not move too far away from the old policy.

3. DDPG

DDPG (Deep Deterministic Policy Gradient) is an actor-critic reinforcement learning algorithm designed mainly for **continuous action spaces**.

It uses:

- Actor network
- Critic network
- Target networks
- Experience replay

In simple words:
DDPG learns which exact continuous action should be taken in a given state.

Code:

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

from stable_baselines3 import PPO, DDPG
from sb3_contrib import TRPO
def test_model(model, env, episodes=10):

  rewards = []

  for episode in range(episodes):

    obs, info = env.reset()
    done = False
    total_reward = 0

    while not done:

      action, _ = model.predict(obs,deterministic=True )

      obs, reward, terminated, truncated, info = env.step(action)

      total_reward += reward

      done = terminated or truncated

    rewards.append(total_reward)
```

```
    return np.mean(rewards)

print("Training PPO...")

ppo_env = gym.make("Pendulum-v1")

ppo_model = PPO("MlpPolicy",ppo_env,verbose=0)

ppo_model.learn(total_timesteps=10000)

ppo_reward = test_model( ppo_model,ppo_env)

ppo_env.close()

print("PPO Average Reward:", ppo_reward)

print("\nTraining TRPO...")

trpo_env = gym.make("Pendulum-v1")

trpo_model = TRPO(
    "MlpPolicy",
    trpo_env,
    verbose=0
)

trpo_model.learn(
    total_timesteps=10000
)

trpo_reward = test_model(
    trpo_model,
    trpo_env
)

trpo_env.close()

print("TRPO Average Reward:", trpo_reward)

print("\nTraining DDPG...")

ddpg_env = gym.make("Pendulum-v1")
```

```
ddpg_model = DDPG(
   "MlpPolicy",
   ddpg_env,
   verbose=0
)

ddpg_model.learn(
   total_timesteps=10000
)

ddpg_reward = test_model(
   ddpg_model,
   ddpg_env
)

ddpg_env.close()

print("DDPG Average Reward:", ddpg_reward)

print("\n==============================")
print("     PERFORMANCE COMPARISON")
print("==============================")

print("PPO  :", round(ppo_reward, 2))
print("TRPO :", round(trpo_reward, 2))
print("DDPG :", round(ddpg_reward, 2))

algorithms = ["PPO", "TRPO", "DDPG"]

rewards = [
   ppo_reward,
   trpo_reward,
   ddpg_reward
]

plt.figure(figsize=(8, 5))

plt.bar(
   algorithms,
   rewards
)
plt.xlabel("Algorithm")
plt.ylabel("Average Reward")
```

```
plt.title(
   "PPO vs TRPO vs DDPG Performance"
)
plt.grid(axis="y")
plt.show()
```

Output:

```
Training PPO...
PPO Average Reward: -1232.988159900445

Training TRPO...
TRPO Average Reward: -1135.8420332590592

Training DDPG...
DDPG Average Reward: -117.4629906408589

==============================
     PERFORMANCE COMPARISON
==============================
PPO  : -1232.99
TRPO : -1135.84
DDPG : -117.46
```